

2025/2026

Cardify

Technical Documentation

Ingegneria del Software e Progettazione Web (ISPW)

Yari Antonio Bellardini : 0309879

Nota: Questa relazione ha ottenuto la valutazione di 30/30. Il voto conseguito non ne indica l'assenza di errori. Vi consiglio di considerarla solo come un esempio da affiancare al resto del materiale disponibile sul Drive.

INDICE

1	Software Requirement Specification	3
1.1	Introduction	3
1.1.1	Aim of the document	3
1.1.2	Overview of the defined system	3
1.1.3	Hardware and Software Requirements	3
1.1.4	Related Systems, Pros and Cons	4
2	User Stories	5
3	Functional Requirements	5
4	Use Cases	6
4.1	Overview Diagram	6
4.2	Internal Steps	7
4.2.1	Use Case: Trade	7
5	Storyboards	8
6	Design	10
6.1	Class Diagram	10
6.1.1	VOPC (Analysis)	10
6.1.2	Design-level Diagram	11
6.2	Design Patterns	13
6.3	Activity Diagram	14
6.4	Sequence Diagram	15
6.5	State Diagram	16
7	Testing	16
7.1	LoginControllerTest:	16
7.2	CollectionControllerTest:	16
7.3	TradeControllerTest:	16
8	Github Repository	17
8.1	Project link	17
9	SonarCloud Report	17
9.1	Project link	17
10	Video	17
10.1	Link youtube	17

1 Software Requirement Specification

1.1 Introduction

1.1.1 Aim of the document

The primary objective of this document is to objectively define the fundamental and specific requirements for the system under development. It serves as the official guide for the development team, comprehensively reviewing all critical internal and external aspects of the final solution.

1.1.2 Overview of the defined system

Cardify is a comprehensive management and trading platform designed for Trading Card Game enthusiasts and local game stores. The application allows collectors to organize their card collections into digital binders and browse a vast database powered by the TCGdex API. The system enables users to create and manage trade proposals, facilitating card exchanges between collectors. To ensure transaction safety, meet-ups can be scheduled exclusively at official partner shops. A key feature is the integrated trade session system, which handles real-time presence verification and arrival notifications to ensure secure transactions. Cardify provides a dual-interface experience, catering to both individual collectors and shop owners with specialized dashboards.

1.1.3 Hardware and Software Requirements

Software:

- **Java Runtime:** JDK 25 or higher.
- **Database:** MySQL Server 8.0+ or 9.0+.
- **Operating System:**
 - Windows 10/11
 - macOS (Intel/M-series)
 - Linux (any distribution supporting Java 25)
- **Internet Connection:** Active (required for retrieving card data via the TCGdex external API).

Hardware :

- **Processor:** Multi-core CPU (Intel i3 / AMD Ryzen 3 or higher) clocked at 2.0 GHz.
- **RAM:** 4 GB recommended (minimum 2 GB dedicated).
- **Disk Space:** 200 MB of free space.
- **Display Resolution:** 1280x720 or higher (required for correct rendering of the JavaFX graphical user interface).

1.1.4 Related Systems, Pros and Cons

System Name	Pros	Cons
Cardmarket	Extensive card database, establishes the standard for market pricing, robust feedback system for security.	Strictly sales-oriented, complex and outdated User Interface, transaction fees apply and isolate users behind screens, bypassing local communities and physical stores
Social & Informal Channels (e.g., Facebook Marketplace, Discord Servers, WhatsApp Groups)	No listing fees or commissions. Often organized by city or region, allowing for potential hand-delivery. Facilitates direct human communication and negotiation.	Users cannot search within other users' inventories or filter by specific attributes. Lack of user verification, reputation tracking, or third-party mediation makes in-person meetups risky and remote trades unreliable.

2 User Stories

- **US-1:** As a Collector, I want to add trading cards to my digital binders, So that I can organize my inventory and identify duplicates or missing items.
- **US-2:** As a Collector, I want to propose exchange deals to other users by selecting cards from their collection, So that I can trade the cards I don't need for the ones I am missing.
- **US-3:** As a Store Manager, I want to publish new events, So that I can attract customers to visit my store.

3 Functional Requirements

- **FR-1:** The system shall display a list of cards that are missing from the user's collection.
- **FR-2:** The system must organize the user's collection into binders divided by set.
- **FR-3:** The system shall provide a list of registered shops where collectors can make their trades.

4 Use Cases

4.1 Overview Diagram

I casi d'uso con * includono il login

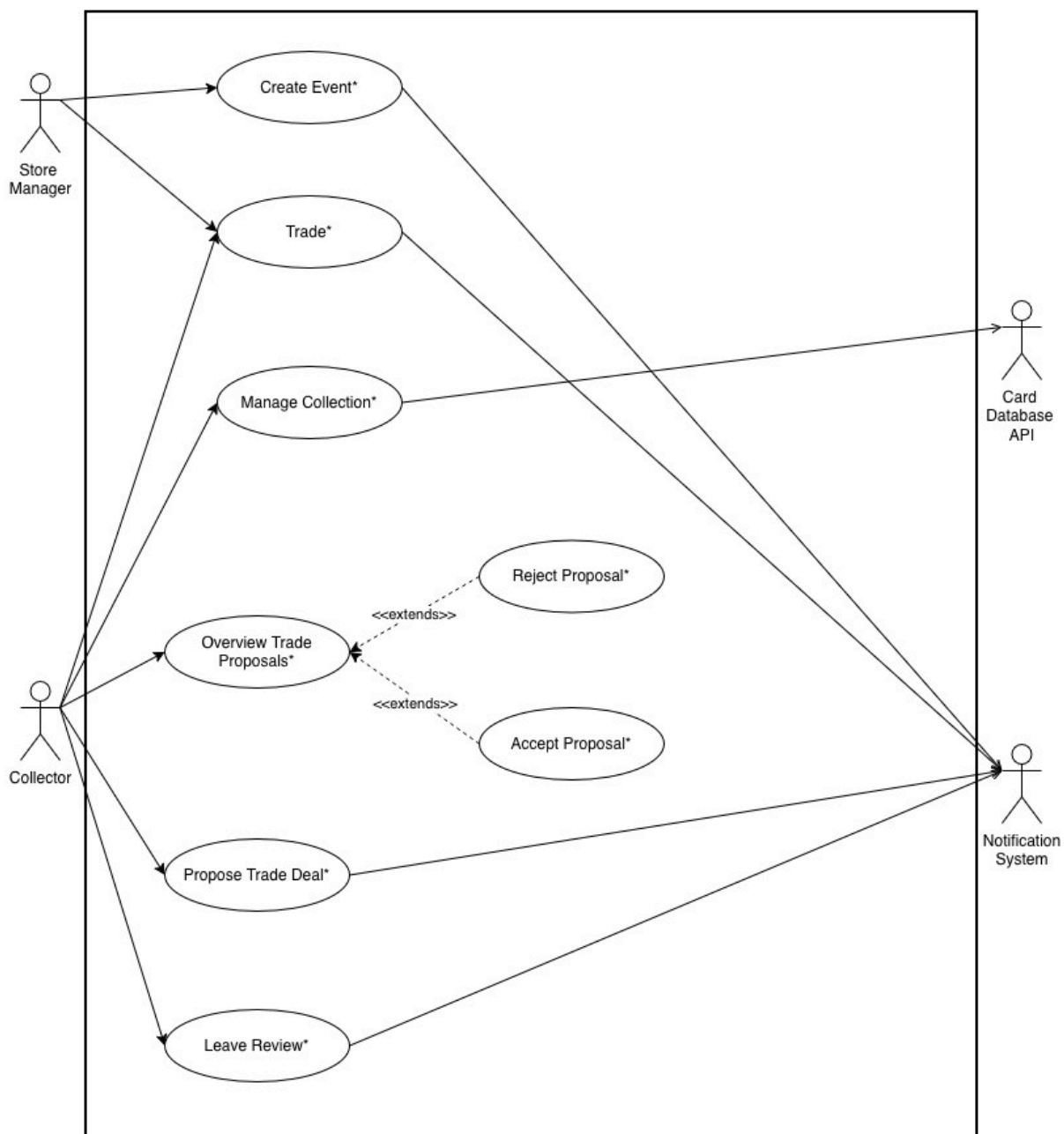


Figura 1: Use Case Diagram

4.2 Internal Steps

4.2.1 Use Case: Trade

1. **Collector A** requests presence verification.
2. The **System** provides a verification code for **Collector A**.
3. The **Store Manager** validates **Collector A**'s presence using the provided code.
4. The **System** verifies the code and notifies **Collector B**.
5. **Collector B** requests presence verification.
6. The **System** provides a verification code for **Collector B**.
7. The **Store Manager** validates **Collector B**'s presence using the provided code.
8. The **System** verifies the code and enables the inspection phase.
9. The **Store Manager** initiates the physical card inspection.
10. The **System** presents the inspection items.
11. The **Store Manager** certifies that the inspection is successful for both parties.
12. The **Store Manager** authorizes the trade completion.
13. The **System** transfers the ownership of the digital cards between the two collectors and records the trade as complete.

4.2.1.1 Extensions

4a. Invalid Code

1. The **System** notifies the **Store Manager** of the invalid code.
2. Return to step 3.

4b. Collector Timeout

1. The **System** detects that no valid code has been provided for **Collector A** within the expected time frame and notifies the **Store Manager**.
2. The **Store Manager** cancels the trade.
3. The **System** notifies both parties.

11a. Inspection Failed

1. The **Store Manager** records an inspection issue.
2. The **System** terminates the trade and notifies both parties.

5 Storyboards

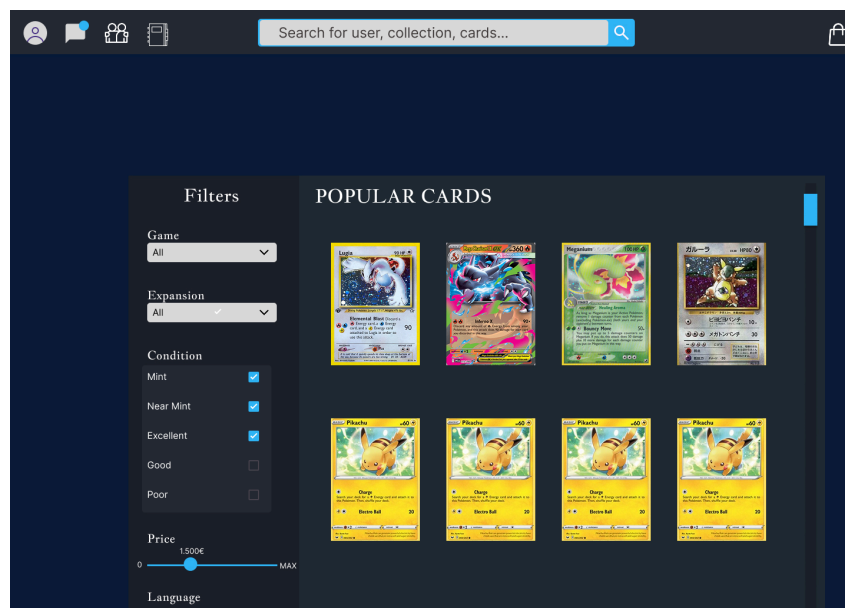


Figura 2: Collector's Homepage

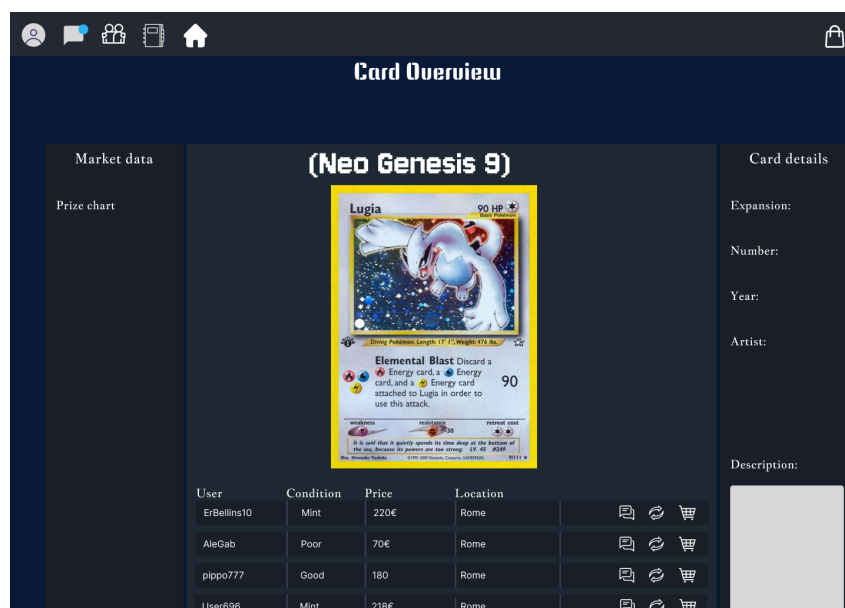


Figura 3: Card Overview page

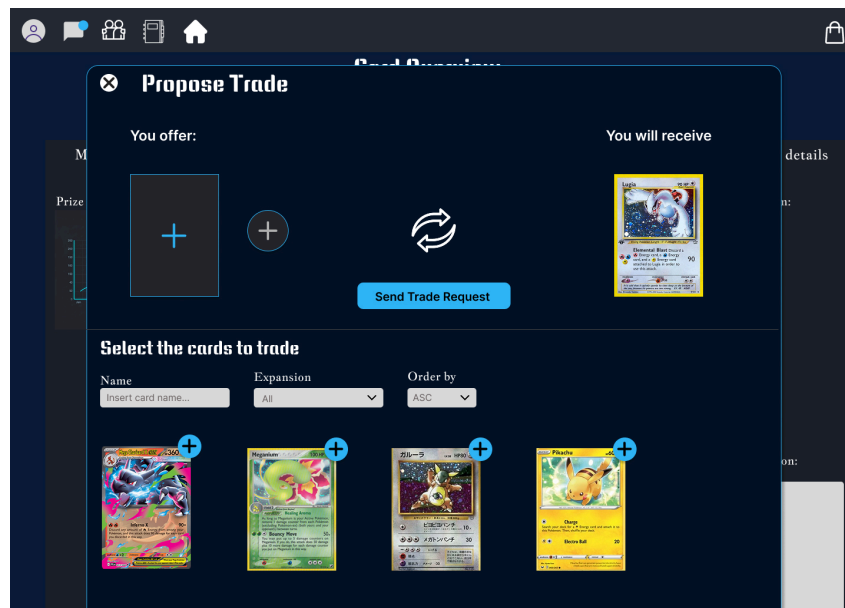


Figura 4: Trade proposal interface

,

6 Design

6.1 Class Diagram

6.1.1 VOPC (Analysis)

This BCE diagram was drafted during the analysis phase. The final implementation details may vary from those shown in this diagram, as they result from subsequent design refinements.

⚠ ATTENZIONE: Questo VOPC molto probabilmente non è ottimale e andrebbe ristrutturato. Vi consiglio di non realizzarlo come riferimento, ma di capire approfonditamente a lezione il perchè e come si realizza questo diagramma, in quanto molto probabilmente dovrete discuterlo in sede di esame orale.

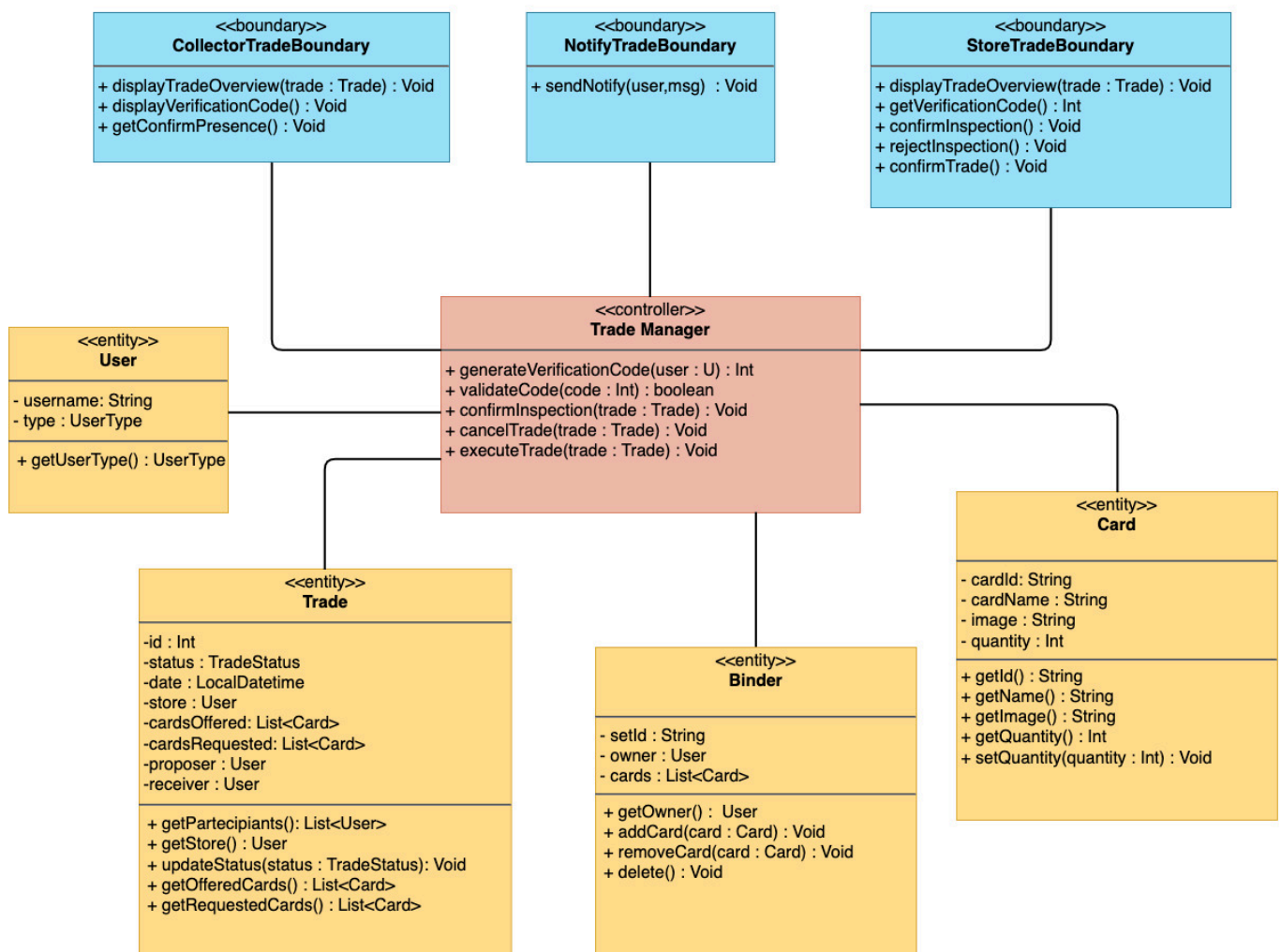


Figura 5: BCE View Of Participating Classes

6.1.2 Design-level Diagram

6.1.2.1 View Abstract Factory

To maintain diagrammatic conciseness, only the core domain entities are depicted in this representation. It is implied that the omitted classes adhere to an identical architectural structure.

⚠ DISCLAIMER: La ViewFactory dovrebbe essere astratta (e con un operazione getFactory()), non un'interfaccia.

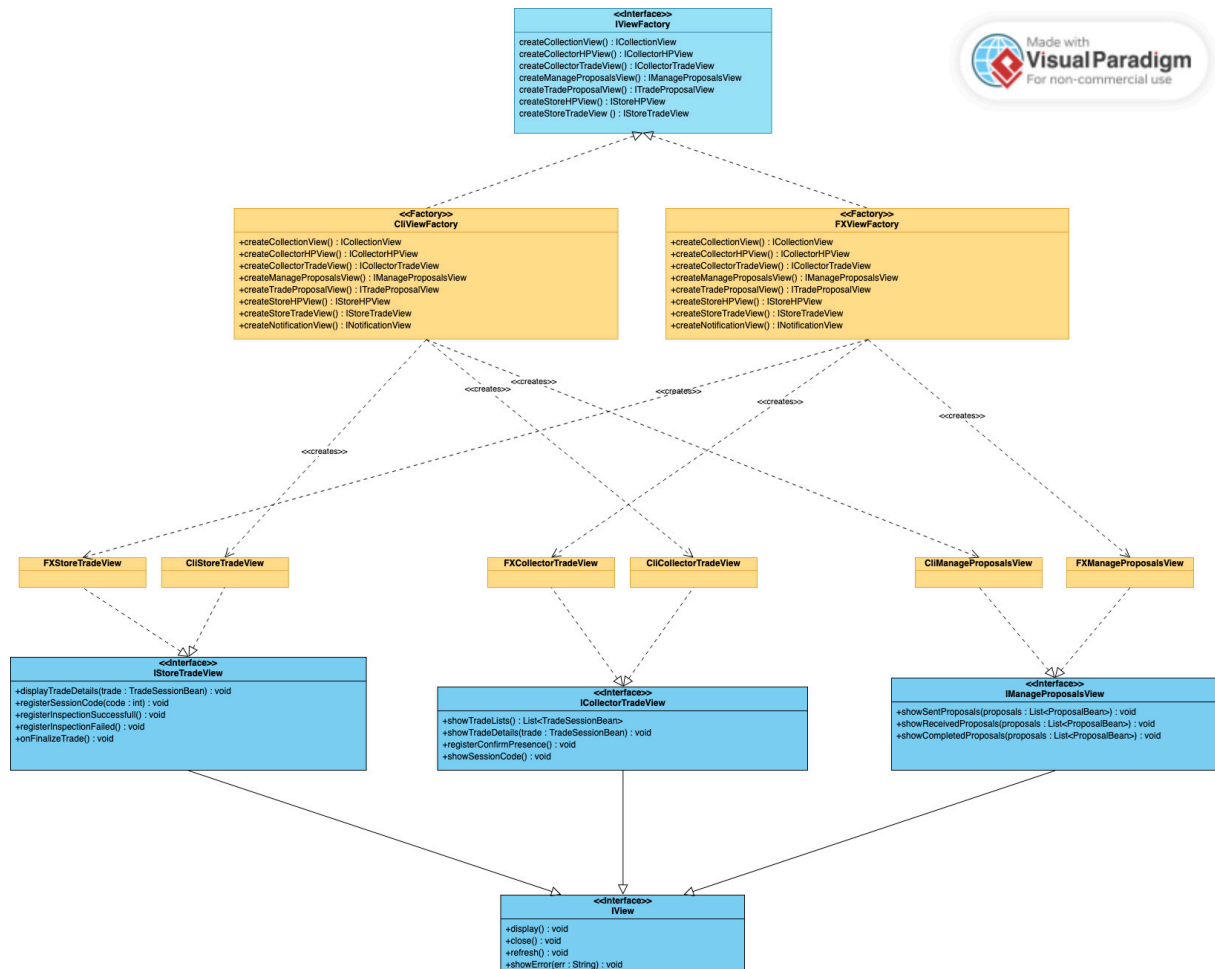


Figura 6: View Abstract Factory

6.1.2.2 DAO Abstract Factory

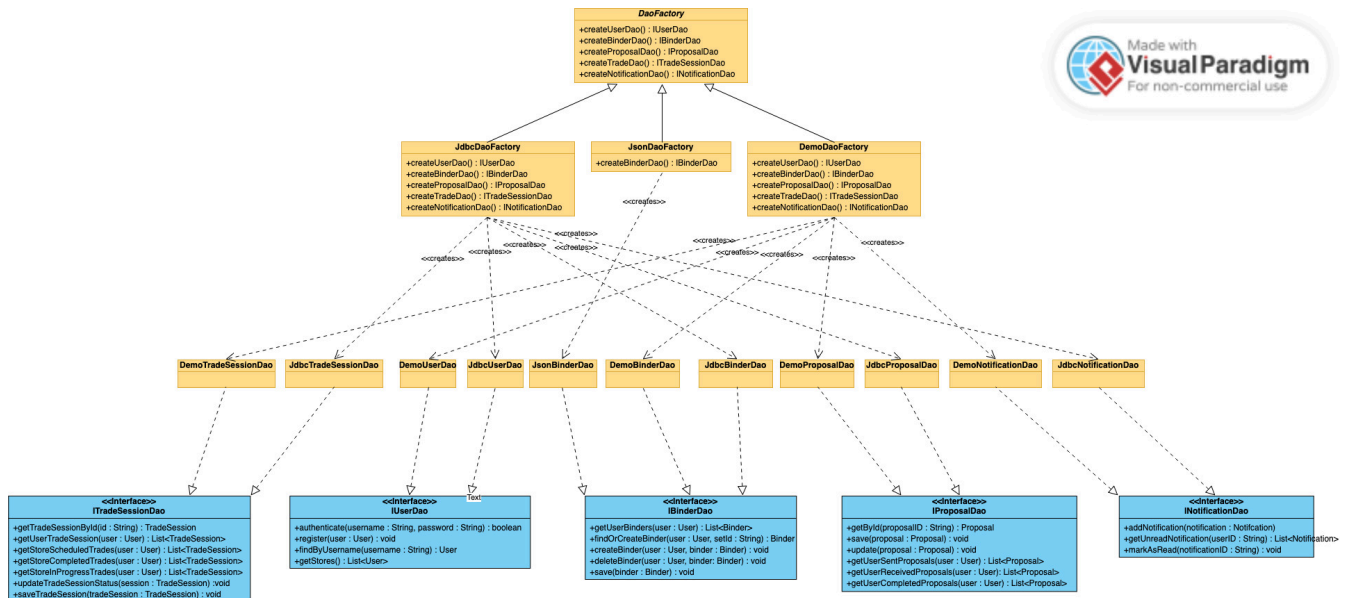


Figura 7: DAO Abstract Factory

6.1.2.3 Card Provider Factory

To ensure future growth, the system employs a modular design managed by an ApiFactory. While the current implementation focuses on Pokémon via **PokemonCardProvider**, the abstraction ensures that new card types can be easily added as distinct providers.

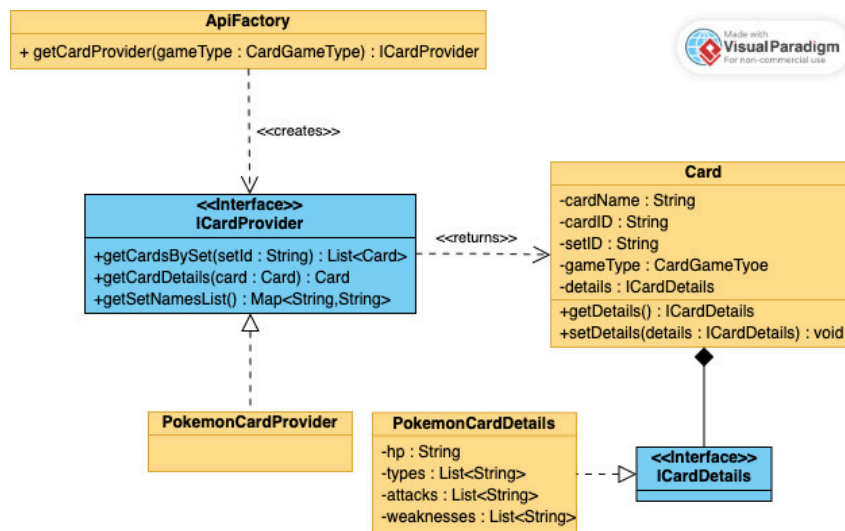


Figura 8: Card Provider Factory

6.1.2.4 DB Connector Singleton

DBConnector
- instance : DBConnector - url : String - user : String - password : String - driver : String
+ getConnection() : Connection - getInstance() : DBConnector

Figura 9: DB Connector Singleton

6.2 Design Patterns

The system architecture follows the **Model-View-Controller (MVC)** pattern to promote maintainability and separation of concerns, utilizing an **Abstract Factory** to manage GUI families (JavaFX and CLI) so that controllers depend only on abstractions. Regarding data management, the system utilizes **DAO** interfaces combined with an **Abstract Factory** to render the application agnostic to the storage mechanism; the persistence strategy, such as **JDBC** for MySQL or **Demo** for in-memory storage, is selected at runtime via configuration. To isolate the Domain model, **Mappers** translate internal entities into **Beans**, ensuring the View layer interacts only with Beans Classes. External API integration is handled by a **Factory Method** (ApiFactory) that creates CardProviders, which act as **Adapters** to translate external service results into a unified internal format. A notification system is implemented using the **Observer** pattern, where controllers act as **Subjects** to notify the NotificationService of business events like trade proposals. The **Singleton** pattern is applied to the DBConnector for unique database access.

6.3 Activity Diagram

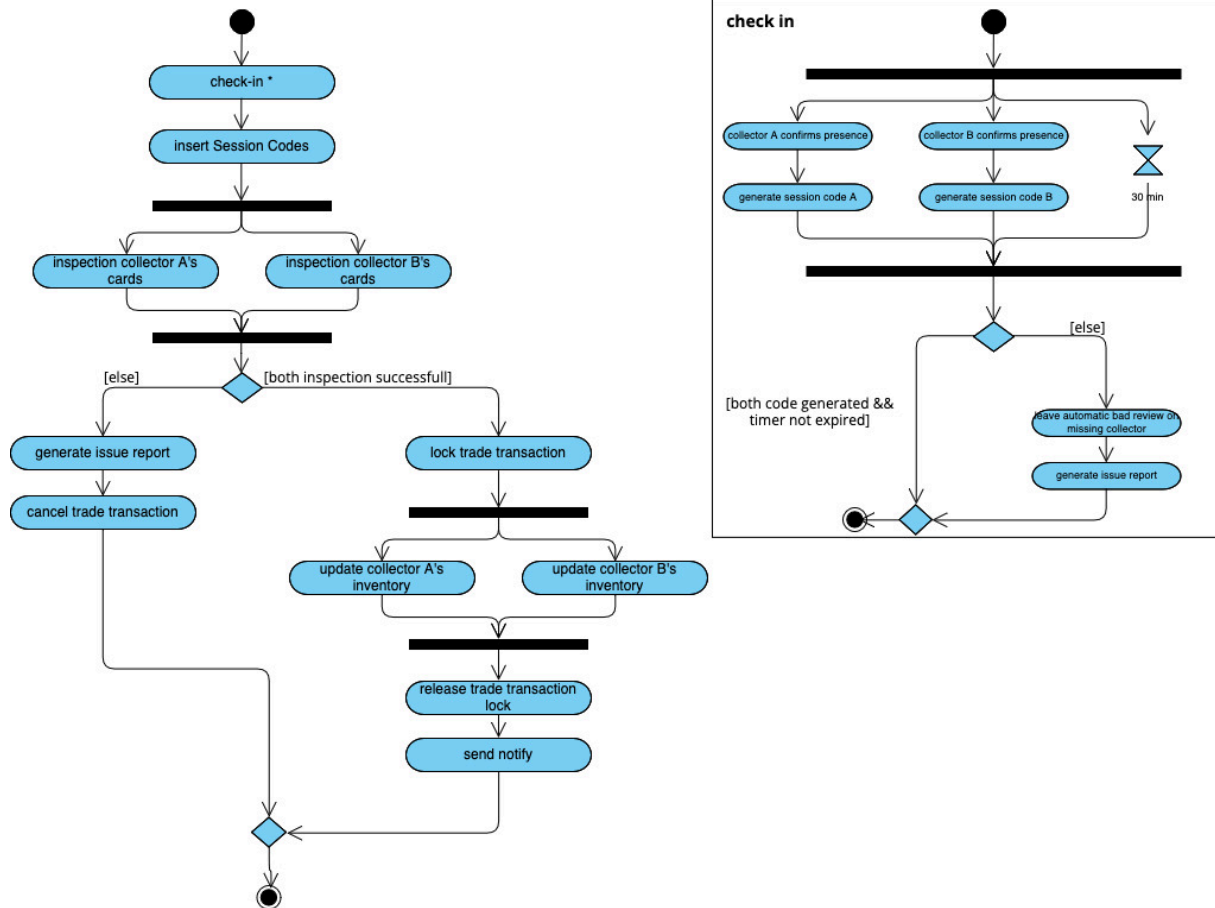


Figura 10: Activity Diagram

6.4 Sequence Diagram

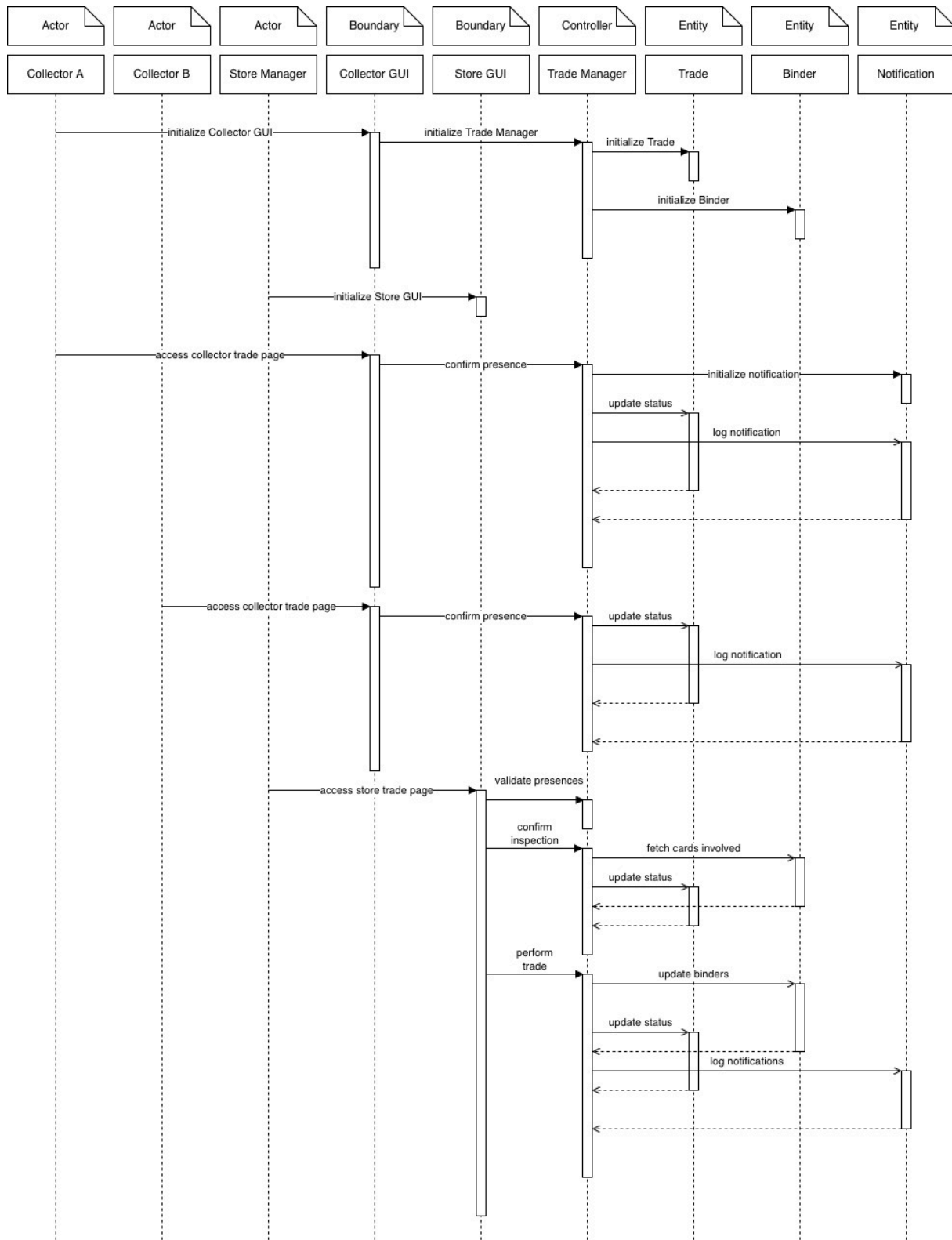


Figura 11: Sequence Diagram

6.5 State Diagram

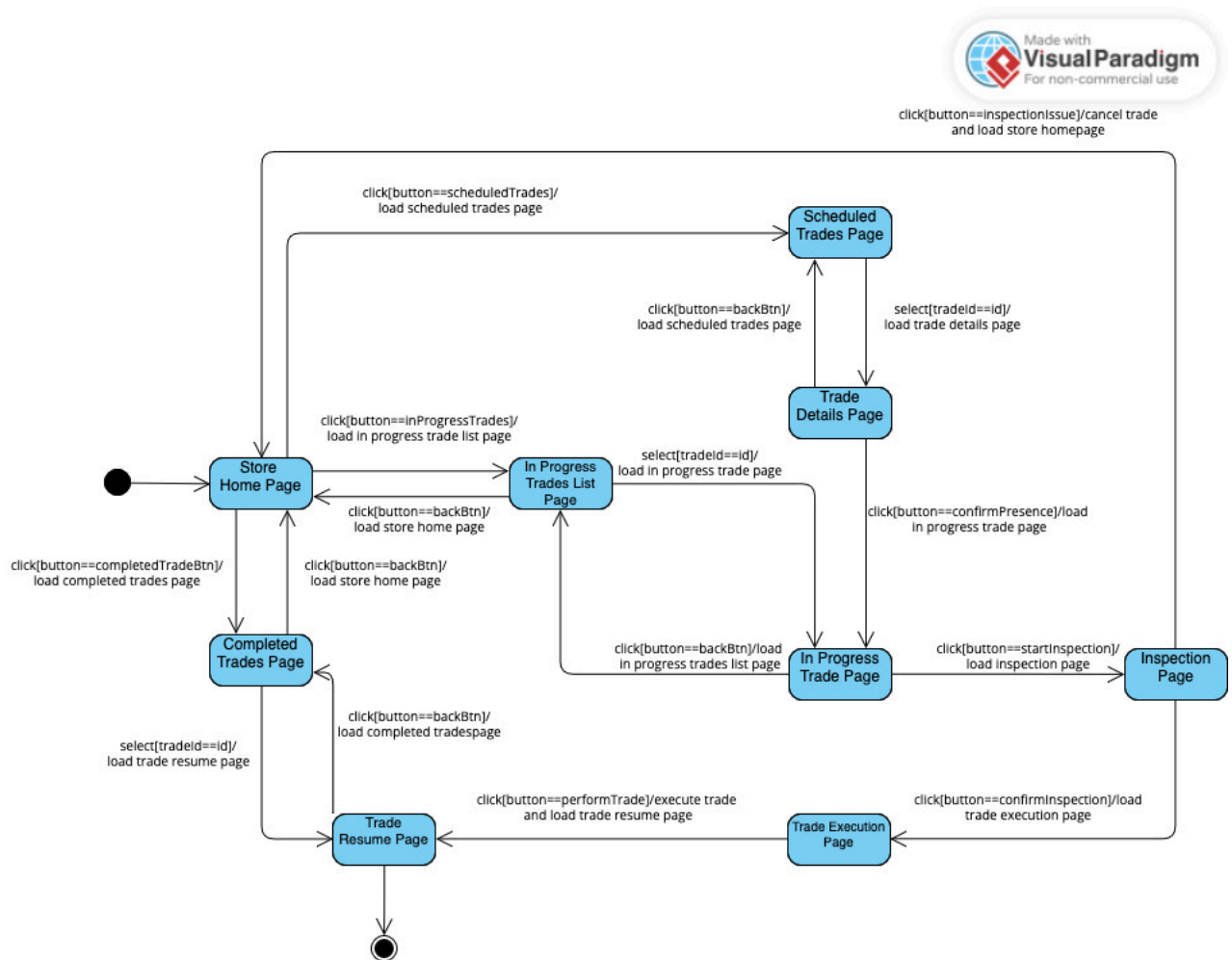


Figura 12: State Diagram

7 Testing

7.1 LoginControllerTest:

This class verifies the authentication and registration flows of the application. It ensures users can log in with valid credentials, handles invalid attempts with clear error feedback, and validates that new user accounts are correctly created and persisted in the system.

7.2 CollectionControllerTest:

This test suite focuses on the management of card collections and binders. It ensures that users cannot create duplicate binders for the same set.

7.3 TradeControllerTest:

This class validates the core logic involved in card trading between users. It confirms that transfers are executed accurately, quantities are correctly updated for both participants, and new binders are automatically generated when users receive cards from sets they don't yet own.

8 Github Repository

8.1 Project link

<https://github.com/Yari17/Cardify>

9 SonarCloud Report

9.1 Project link

https://sonarcloud.io/project/overview?id=Yari17_Cardify

10 Video

10.1 Link youtube

<https://youtu.be/8GuhOFJs604>